

AdPlatform/DAADD — Technical Guide for Developers

Platform: AdPlatform/DAADD (Two-Sided AdTech Platform)
Role: Developer (API Integration, Third-Party Tooling, Custom Implementations)
Last Updated: May 2026
Audience: Backend engineers, full-stack developers, platform integrators

Table of Contents

- 1. [Getting Started](#)
 - 2. [Authentication & Authorization](#)
 - 3. [Core API Patterns](#)
 - 4. [Campaign Management API](#)
 - 5. [Event Tracking API](#)
 - 6. [Analytics & Reporting API](#)
 - 7. [Webhooks & Real-Time Events](#)
 - 8. [OAuth Integration](#)
 - 9. [Conversion Pixel Implementation](#)
 - 10. [Error Handling & Retry Logic](#)
 - 11. [Rate Limiting](#)
 - 12. [Code Examples](#)
 - 13. [Testing & Debugging](#)
 - 14. [API Reference](#)
-

Getting Started

Environment Setup

Base URLs:

```
# Production
API_BASE_URL=https://adplatform.example.com/api/v1
WEBHOOK_DOMAIN=https://adplatform.example.com

# Staging (for testing)
API_BASE_URL_STAGING=https://staging.adplatform.example.com/api/v1

# Local Development (if running locally)
API_BASE_URL_LOCAL=http://localhost:4000/api/v1
```

Required Headers

All API requests must include:

```
Authorization: Bearer <JWT_TOKEN>
Content-Type: application/json
User-Agent: YourApp/1.0 (+https://yoursite.com)
X-Request-ID: <unique-id> # Optional but recommended for debugging
```

Example:

```
curl -X GET https://adplatform.example.com/api/v1/campaigns \
-H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." \
-H "Content-Type: application/json" \
-H "User-Agent: MyIntegration/1.0"
```

API Response Format

All responses follow a consistent envelope:

```
{
  "success": true,
  "data": { /* response data */ },
  "meta": {
    "timestamp": "2026-05-17T14:30:00Z",
    "request_id": "req_abc123",
    "api_version": "v1"
  }
}
```

Error response:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid campaign budget",
    "details": {
      "budget_total": "Must be a positive number"
    }
  },
  "meta": {
    "timestamp": "2026-05-17T14:30:00Z",
    "request_id": "req_abc123"
  }
}
```

Authentication & Authorization

1. OAuth 2.0 Authorization Code Flow

For third-party integrations (your app requests access to user's AdPlatform account):

Step 1: Redirect user to AdPlatform login

```

const clientId = 'your_client_id';
const redirectUri = 'https://yourapp.com/auth/adplatform/callback';
const scope = 'campaigns:read campaigns:write analytics:read webhooks:write';

const authUrl = `https://adplatform.example.com/oauth/authorize?` +
  `client_id=${clientId}&` +
  `redirect_uri=${encodeURIComponent(redirectUri)}&` +
  `scope=${encodeURIComponent(scope)}&` +
  `response_type=code`;

window.location.href = authUrl;

```

Step 2: User logs in and grants permission

User is redirected back to your app with an authorization code:

```
https://yourapp.com/auth/adplatform/callback?code=auth_code_123&state=random_state
```

Step 3: Exchange code for access token

```

// On your backend
const response = await fetch('https://adplatform.example.com/api/v1/oauth/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    grant_type: 'authorization_code',
    code: 'auth_code_123',
    client_id: 'your_client_id',
    client_secret: 'your_client_secret',
    redirect_uri: 'https://yourapp.com/auth/adplatform/callback'
  })
});

const { access_token, refresh_token, expires_in } = await response.json();
// Store access_token (use for API calls)
// Store refresh_token (use to get new access token when expired)

```

Step 4: Use token to make API calls

```

const response = await fetch('https://adplatform.example.com/api/v1/campaigns', {
  headers: {
    'Authorization': `Bearer ${access_token}`
  }
});

```

2. Service-to-Service Authentication

If your backend needs to call AdPlatform API on behalf of your service (not a user):

Use Client Credentials Flow:

```
// Exchange credentials for a service token
const response = await fetch('https://adplatform.example.com/api/v1/oauth/token', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({
    grant_type: 'client_credentials',
    client_id: 'your_client_id',
    client_secret: 'your_client_secret',
    scope: 'campaigns:read analytics:read'
  })
});

const { access_token, expires_in } = await response.json();
// Token valid for ~1 hour; cache and refresh as needed
```

3. Scopes & Permissions

Available scopes (combine with spaces):

Scope	Allows	Use Case
campaigns:read	Read campaigns	View campaign details
campaigns:write	Create/edit campaigns	Programmatic campaign creation
analytics:read	Read analytics	Dashboard integrations
analytics:write	Export/process analytics	BI platform integrations
events:write	Log events (impressions, clicks)	Tracker implementations
webhooks:write	Register/manage webhooks	Event notification setup
users:read	Read user info	User profile access
ai:read	Read AI recommendations	Optimization dashboards

Request specific scopes:

```
const scope = 'campaigns:read analytics:read events:write';
```

Core API Patterns

Pagination

List endpoints support pagination:

```
GET /api/v1/campaigns?page=1&limit=20&sort=-created_at
```

Query Parameters:

Param	Type	Default	Description
page	number	1	Page number (1-indexed)
limit	number	20	Items per page (max 100)
sort	string	-created_at	Sort field; prefix - for descending

Response:

```
{
  "success": true,
  "data": [ /* items */ ],
  "pagination": {
    "total": 47,
    "page": 1,
    "limit": 20,
    "pages": 3,
    "has_more": true
  }
}
```

Filtering

Use query parameters to filter results:

```
# Filter by status
GET /api/v1/campaigns?status=ACTIVE

# Multiple filters
GET /api/v1/campaigns?status=ACTIVE&created_after=2026-05-01&created_before=2026-05-31

# Filter with range
GET /api/v1/campaigns?budget_min=1000&budget_max=5000
```

Partial Updates

Use PATCH (not PUT) for partial updates:

```
# Only updates name; other fields unchanged
PATCH /api/v1/campaigns/camp_456 \
  -d '{"name": "New Name"}'

# vs. PUT (replaces entire campaign)
# Never use PUT; always use PATCH
```

Conditional Requests

For safety, use ETags to prevent overwriting stale data:

```
# Initial GET returns ETag
curl -X GET https://adplatform.example.com/api/v1/campaigns/camp_456
# Response header: ETag: "abc123"

# Later, use ETag in If-Match
curl -X PATCH https://adplatform.example.com/api/v1/campaigns/camp_456 \
  -H 'If-Match: "abc123"' \
  -d '{"name": "Updated"}'

# If resource changed since ETag, returns 412 Precondition Failed
```

Campaign Management API

Create Campaign

POST /api/v1/campaigns

Request:

```
{
  "name": "Summer Sale 2026",
  "description": "Promote summer collection",
  "budget_total": 5000,
  "start_date": "2026-06-01T00:00:00Z",
  "end_date": "2026-08-31T23:59:59Z",
  "platforms": ["web", "mobile"],
  "targeting": {
    "countries": ["US", "UK"],
    "min_age": 18,
    "max_age": 65,
    "genders": ["all"],
    "interests": ["shopping", "fashion"]
  },
  "creative": {
    "title": "Exclusive Summer Deals",
    "description": "Get up to 50% off",
    "image_url": "https://cdn.example.com/image.jpg"
  },
  "reward_value": 0.50,
  "status": "DRAFT"
}
```

Response (201 Created):

```
{
  "success": true,
  "data": {
    "campaign_id": "camp_456",
    "name": "Summer Sale 2026",
    "status": "DRAFT",
    "created_at": "2026-05-17T10:30:00Z"
  }
}
```

List Campaigns

GET /api/v1/campaigns?page=1&limit=20&status=ACTIVE

Get Campaign Details

GET /api/v1/campaigns/camp_456

Response:

```
{
  "success": true,
  "data": {
    "campaign_id": "camp_456",
    "advertiser_id": "user_123",
    "name": "Summer Sale 2026",
    "status": "ACTIVE",
    "budget_total": 5000,
    "budget_spent": 1250.50,
    "impressions": 45000,
    "clicks": 890,
    "conversions": 45,
    "roi": 485,
    "created_at": "2026-05-17T10:30:00Z",
    "updated_at": "2026-05-17T14:30:00Z"
  }
}
```

Update Campaign

```
PATCH /api/v1/campaigns/camp_456
```

Request (only changed fields):

```
{
  "name": "Summer Sale Extended",
  "budget_total": 7500,
  "end_date": "2026-09-30T23:59:59Z"
}
```

Clone Campaign

```
POST /api/v1/campaigns/camp_456/clone
```

Request:

```
{
  "name": "Summer Sale 2026 - Variant B"
}
```

Change Campaign Status

```
PATCH /api/v1/campaigns/camp_456
```

```
# Request
```

```
{
  "status": "ACTIVE" // or "PAUSED", "COMPLETED", "ARCHIVED"
}
```

Event Tracking API

Track Single Event

POST /api/v1/events/track

Request:

```
{
  "event_type": "click",
  "campaign_id": "camp_456",
  "ad_id": "ad_789",
  "user_id": "user_123",
  "device_type": "mobile",
  "ip_address": "203.0.113.45",
  "geo_lat": 37.7749,
  "geo_lng": -122.4194,
  "geo_city": "San Francisco",
  "geo_country": "US",
  "metadata": {
    "referrer": "google.com",
    "utm_source": "email",
    "utm_campaign": "summer_sale"
  }
}
```

Event Types:

- impression — User saw the ad
- click — User clicked the ad
- conversion — User completed action (purchase, signup)
- viewable_impression — Ad was in viewport for 1+ second

Track Batch Events

POST /api/v1/events/track/batch

Request:


```
{
  "events": [
    {
      "event_type": "impression",
      "campaign_id": "camp_456",
      "ad_id": "ad_789",
      "user_id": "user_123"
    },
    {
      "event_type": "click",
      "campaign_id": "camp_456",
      "ad_id": "ad_789",
      "user_id": "user_123"
    },
    {
      "event_type": "conversion",
      "campaign_id": "camp_456",
      "ad_id": "ad_789",
      "user_id": "user_456"
    }
  ]
}
```

Ideal for:

- Server-side tracking of multiple events in one request
- Reduces API calls and improves efficiency
- Useful for batch processors or data pipelines

Analytics & Reporting API

Get Campaign Dashboard

```
GET /api/v1/analytics/dashboard/camp_456?start_date=2026-05-01&end_date=2026-05-31
```

Response:

```

{
  "success": true,
  "data": {
    "campaign_id": "camp_456",
    "period": "2026-05-01 to 2026-05-31",
    "metrics": {
      "impressions": 45000,
      "clicks": 890,
      "click_through_rate": 1.98,
      "conversions": 45,
      "conversion_rate": 5.06,
      "total_spent": 2145.75,
      "cost_per_conversion": 47.68,
      "revenue": 12500,
      "roi": 483,
      "viewable_impressions": 38250,
      "viewability_rate": 85
    },
    "timeseries": [
      {
        "date": "2026-05-10",
        "impressions": 3200,
        "clicks": 65,
        "conversions": 3,
        "spent": 147.50
      }
    ],
    "top_devices": [
      { "device": "mobile", "clicks": 567, "ctr": 2.15 },
      { "device": "desktop", "clicks": 323, "ctr": 1.75 }
    ],
    "top_regions": [
      { "country": "US", "clicks": 650, "conversions": 35 },
      { "country": "UK", "clicks": 240, "conversions": 10 }
    ]
  }
}

```

Export Analytics

```
GET /api/v1/analytics/export/camp_456?format=pdf&start_date=2026-05-01&end_date=2026-05-31
```

Formats:

- pdf — Detailed PDF report (embeddable in emails)
- csv — Comma-separated for spreadsheets
- json — Raw JSON for programmatic processing

Response (PDF):

Content-Type: application/pdf
Content-Disposition: attachment; filename="camp_456_report.pdf"
[PDF binary data]

Get Creative Performance

GET /api/v1/analytics/creatives/camp_456

Response:

```
{
  "success": true,
  "data": {
    "creatives": [
      {
        "creative_id": "cr_001",
        "title": "Creative A",
        "is_control": true,
        "impressions": 22500,
        "clicks": 445,
        "ctr": 1.98,
        "conversions": 22,
        "conversion_rate": 4.94
      },
      {
        "creative_id": "cr_002",
        "title": "Creative B",
        "is_control": false,
        "impressions": 22500,
        "clicks": 445,
        "ctr": 1.98,
        "conversions": 23,
        "conversion_rate": 5.17
      }
    ],
    "winner": "cr_002",
    "lift": "4.6%"
  }
}
```

Webhooks & Real-Time Events

Register Webhook

POST /api/v1/webhooks

Request:

```
{
  "url": "https://your-server.com/webhooks/adplatform",
  "secret": "your_webhook_secret_key",
  "events": [
    "campaign.anomaly_detected",
    "campaign.budget_threshold",
    "campaign.auto_paused",
    "campaign.completed",
    "conversion.received"
  ],
  "is_active": true
}
```

Webhook Events

Event	When	Payload
campaign.budget_threshold	Campaign reaches 75%, 90%, 100% spend	campaign_id, threshold, spent, total
campaign.anomaly_detected	Anomaly detected (e.g., sudden CTR drop)	campaign_id, anomaly_type, severity, description
campaign.auto_paused	Campaign auto-paused due to anomaly	campaign_id, reason, timestamp
campaign.completed	Campaign budget exhausted or end date passed	campaign_id, reason
conversion.received	New conversion via pixel	campaign_id, conversion_value, source

Receiving Webhooks

Your server receives **HTTPS POST** with signed payload:

```
POST /webhooks/adplatform HTTP/1.1
Host: your-server.com
Content-Type: application/json
X-AdPlatform-Signature: sha256=abcdef1234567890
X-AdPlatform-Timestamp: 1715953800
X-AdPlatform-Event: campaign.budget_threshold

{
  "event": "campaign.budget_threshold",
  "timestamp": "2026-05-17T14:30:00Z",
  "data": {
    "campaign_id": "camp_456",
    "campaign_name": "Summer Sale",
    "threshold": 75,
    "budget_spent": 3750,
    "budget_total": 5000
  }
}
```

Verify Webhook Signature

```

const crypto = require('crypto');

function verifyWebhookSignature(payload, signature, secret) {
  const expectedSignature = crypto
    .createHmac('sha256', secret)
    .update(JSON.stringify(payload))
    .digest('hex');

  return `sha256=${expectedSignature}` === signature;
}

// In your webhook handler
app.post('/webhooks/adplatform', (req, res) => {
  const signature = req.headers['x-adplatform-signature'];
  const timestamp = req.headers['x-adplatform-timestamp'];

  // Verify signature
  if (!verifyWebhookSignature(req.body, signature, process.env.WEBHOOK_SECRET)) {
    return res.status(401).send('Unauthorized');
  }

  // Verify timestamp (prevent replay attacks)
  const now = Math.floor(Date.now() / 1000);
  if (Math.abs(now - parseInt(timestamp)) > 300) { // 5 minute window
    return res.status(401).send('Request too old');
  }

  // Process webhook
  const event = req.body.event;
  const data = req.body.data;

  console.log(`Received ${event}: ${JSON.stringify(data)}`);

  // Do something with the event
  // ...

  // Return 200 OK immediately (don't hold the connection)
  res.status(200).json({ received: true });
});

```

List Webhooks

```
GET /api/v1/webhooks
```

Delete Webhook

```
DELETE /api/v1/webhooks/wh_123
```

OAuth Integration

Common Use Case: Integrate AdPlatform into Your Dashboard

If you're building a dashboard tool and want to let users connect their AdPlatform accounts:

Step 1: Register Your App

Contact AdPlatform to get:

- client_id
- client_secret
- Approved redirect URIs

Step 2: Send User to AdPlatform Login

```
// In your web app
function connectAdPlatform() {
  const params = new URLSearchParams({
    client_id: 'your_client_id',
    redirect_uri: 'https://yourdash.com/auth/adplatform/callback',
    scope: 'campaigns:read analytics:read',
    response_type: 'code',
    state: generateRandomString() // Security: prevent CSRF
  });

  window.location.href = `https://adplatform.example.com/oauth/authorize?${params}`;
}
```

Step 3: Handle Callback

```
// Your callback endpoint
app.get('/auth/adplatform/callback', async (req, res) => {
  const { code, state } = req.query;

  // Verify state matches session (CSRF protection)
  if (state !== req.session.oauthState) {
    return res.status(400).send('State mismatch');
  }

  // Exchange code for token
  const tokenRes = await fetch('https://adplatform.example.com/api/v1/oauth/token', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      grant_type: 'authorization_code',
      code,
      client_id: process.env.ADPLATFORM_CLIENT_ID,
      client_secret: process.env.ADPLATFORM_CLIENT_SECRET,
      redirect_uri: 'https://yourdash.com/auth/adplatform/callback'
    })
  });

  const { access_token, refresh_token, expires_in } = await tokenRes.json();

  // Store tokens in session/database
  req.session.adplatformToken = {
    access_token,
    refresh_token,
    expires_at: Date.now() + expires_in * 1000
  };

  res.redirect('/dashboard');
});
```

Step 4: Make API Calls

```
// Fetch campaigns
const campaignRes = await fetch('https://adplatform.example.com/api/v1/campaigns', {
  headers: {
    'Authorization': `Bearer ${req.session.adplatformToken.access_token}`
  }
});

const campaigns = await campaignRes.json();
```

Step 5: Refresh Token When Expired

```

async function getValidToken(session) {
  if (Date.now() < session.adplatformToken.expires_at) {
    return session.adplatformToken.access_token; // Still valid
  }

  // Token expired; refresh it
  const refreshRes = await fetch('https://adplatform.example.com/api/v1/oauth/token', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      grant_type: 'refresh_token',
      refresh_token: session.adplatformToken.refresh_token,
      client_id: process.env.ADPLATFORM_CLIENT_ID,
      client_secret: process.env.ADPLATFORM_CLIENT_SECRET
    })
  });

  const { access_token, expires_in } = await refreshRes.json();

  // Update session
  session.adplatformToken.access_token = access_token;
  session.adplatformToken.expires_at = Date.now() + expires_in * 1000;

  return access_token;
}

```

Conversion Pixel Implementation

Server-Side Tracking (Recommended for Merchants)

On your thank-you page after a purchase, POST to AdPlatform:

```

<script>
  // On thank-you page after successful purchase
  const campaignId = 'camp_456'; // From URL param or hidden field
  const userId = 'user_12345'; // Your customer ID (optional)
  const conversionValue = 99.99; // Purchase amount

  fetch(`https://adplatform.example.com/api/v1/pixel/${campaignId}?
uid=${userId}&ev=conversion&val=${conversionValue}`, {
    method: 'POST',
    mode: 'no-cors' // Allow cross-domain
  });
</script>

```

Client-Side Tracking (For Frontend Developers)


```
<!-- Add to any page where you want to track conversions -->
<script>
  window.AdPlatformPixel = {
    track: function(campaignId, eventType, metadata) {
      const url = `https://adplatform.example.com/api/v1/pixel/${campaignId}?ev=${eventType}`;
      const img = new Image();
      img.src = url;
    }
  };

  // Track a purchase
  AdPlatformPixel.track('camp_456', 'conversion', { value: 99.99 });
</script>
```

Query Parameters

Param	Type	Required	Description
uid	string	Optional	Your user ID
ev	string	Required	Event type (e.g., conversion)
val	number	Optional	Conversion value (e.g., purchase amount)
ref	string	Optional	Referrer domain

Example URL:

```
https://adplatform.example.com/api/v1/pixel/camp_456?
uid=cust_123&ev=conversion&val=99.99&ref=yourstore.com
```

Error Handling & Retry Logic

HTTP Status Codes

Code	Meaning	Retry?	Example
200	OK	No	Successful request
201	Created	No	Resource created
400	Bad Request	No	Invalid JSON or missing fields
401	Unauthorized	No	Token expired or invalid
403	Forbidden	No	Don't have permission
404	Not Found	No	Resource doesn't exist
409	Conflict	Conditional	ETag mismatch, try again with new ETag
429	Too Many Requests	Yes	Rate limit; backoff and retry
500	Server Error	Yes	Unexpected error; retry with backoff
503	Service Unavailable	Yes	Server maintenance; retry later

Implement Exponential Backoff

```

async function apiCallWithRetry(url, options, maxRetries = 3) {
  let lastError;

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const response = await fetch(url, options);

      if (response.ok) {
        return response.json();
      }

      // Retry on 429, 500, 502, 503
      if ([429, 500, 502, 503].includes(response.status)) {
        throw new Error(`HTTP ${response.status}`);
      }

      // Don't retry on 4xx (except 429)
      if (response.status >= 400 && response.status < 500) {
        const error = await response.json();
        throw new Error(`${response.status}: ${error.error.message}`);
      }

    } catch (error) {
      lastError = error;

      if (attempt < maxRetries) {
        // Exponential backoff: 2^attempt * 100ms
        const delay = Math.pow(2, attempt) * 100;
        console.log(`Retry ${attempt}/${maxRetries} in ${delay}ms...`);
        await sleep(delay);
      }
    }
  }

  throw lastError;
}

function sleep(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

// Usage
const campaigns = await apiCallWithRetry(
  'https://adplatform.example.com/api/v1/campaigns',
  {
    headers: {
      'Authorization': `Bearer ${token}`
    }
  }
);

```

Handle Specific Errors

```
async function createCampaign(data) {
  try {
    const response = await fetch('https://adplatform.example.com/api/v1/campaigns', {
      method: 'POST',
      headers: {
        'Authorization': `Bearer ${token}`,
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(data)
    });

    if (!response.ok) {
      const error = await response.json();

      switch (error.error.code) {
        case 'VALIDATION_ERROR':
          console.error('Invalid input:', error.error.details);
          throw new Error('Please fix validation errors');

        case 'INSUFFICIENT_BUDGET':
          console.error('Budget exceeds limit');
          throw new Error('Budget too high; max is $10,000');

        case 'UNAUTHORIZED':
          console.error('Token invalid or expired');
          // Redirect to login
          window.location.href = '/login';

        case 'RATE_LIMITED':
          console.error('Rate limited; retry after 60 seconds');
          // Implement queue or retry logic

        default:
          throw new Error(error.error.message);
      }
    }

    return response.json();
  } catch (error) {
    console.error('Campaign creation failed:', error);
    // Log to monitoring service
  }
}
```

Rate Limiting

Limits

Tier	Requests/Minute	Requests/Hour	Concurrent
Free	60	1,000	10
Pro	300	5,000	50
Enterprise	Unlimited	Unlimited	500

Rate Limit Headers

Every response includes:

```
X-RateLimit-Limit: 60
X-RateLimit-Remaining: 45
X-RateLimit-Reset: 1715953800 # Unix timestamp
```

Check Rate Limit Before Requesting

```
async function checkRateLimit(token) {
  const response = await fetch('https://adplatform.example.com/api/v1/rate-limit', {
    headers: {
      'Authorization': `Bearer ${token}`
    }
  });

  const remaining = response.headers.get('X-RateLimit-Remaining');
  const limit = response.headers.get('X-RateLimit-Limit');

  console.log(`${remaining}/${limit} requests remaining`);

  if (remaining < 10) {
    console.warn('Approaching rate limit; slow down requests');
  }
}
```

Code Examples

Example 1: Sync Campaigns to Your DB

```

async function syncCampaignsToDatabase() {
  let page = 1;
  let hasMore = true;

  while (hasMore) {
    const response = await fetch(
      `https://adplatform.example.com/api/v1/campaigns?page=${page}&limit=100`,
      {
        headers: { 'Authorization': `Bearer ${token}` }
      }
    );

    const { data, pagination } = await response.json();

    // Save each campaign to your database
    for (const campaign of data) {
      await db.campaigns.upsert({
        adplatform_campaign_id: campaign.campaign_id,
        name: campaign.name,
        status: campaign.status,
        budget_total: campaign.budget_total,
        budget_spent: campaign.budget_spent,
        roi: campaign.roi,
        synced_at: new Date()
      });
    }

    hasMore = pagination.has_more;
    page++;
  }

  console.log('Campaign sync complete');
}

```

Example 2: Daily Analytics Report

```

async function sendDailyReport(campaignId) {
  const yesterday = new Date(Date.now() - 24*60*60*1000).toISOString().split('T')[0];

  const response = await fetch(
    `https://adplatform.example.com/api/v1/analytics/dashboard/${campaignId}?
start_date=${yesterday}&end_date=${yesterday}`,
    {
      headers: { 'Authorization': `Bearer ${token}` }
    }
  );

  const { data } = await response.json();
  const metrics = data.metrics;

  // Send to Slack
  await fetch(process.env.SLACK_WEBHOOK, {
    method: 'POST',
    body: JSON.stringify({
      text: `    Daily Report: ${data.name}`,
      blocks: [
        {
          type: 'section',
          text: {
            type: 'mrkdwn',
            text: `*Impressions:* ${metrics.impressions}\n` +
              `*Clicks:* ${metrics.clicks} (CTR: ${metrics.click_through_rate.toFixed(2)}%)\n` +
              `*Conversions:* ${metrics.conversions} (CPA: $$${metrics.cost_per_conversion.toFixed(2)})\n` +
              `*Spent:* $$${metrics.total_spent.toFixed(2)}\n` +
              `*ROI:* ${metrics.roi}%`
          }
        }
      ]
    })
  });
}

// Run daily at 9 AM
cron.schedule('0 9 * * *', () => {
  sendDailyReport('camp_456');
});

```

Example 3: Implement Webhook Handler

```

const express = require('express');
const crypto = require('crypto');
const app = express();

app.use(express.json());

// Webhook secret from AdPlatform
const WEBHOOK_SECRET = process.env.ADPLATFORM_WEBHOOK_SECRET;

```

```
function verifySignature(payload, signature) {
  const hash = crypto
    .createHmac('sha256', WEBHOOK_SECRET)
    .update(JSON.stringify(payload))
    .digest('hex');

  return `sha256=${hash}` === signature;
}

app.post('/webhooks/adplatform', (req, res) => {
  const signature = req.headers['x-adplatform-signature'];
  const timestamp = req.headers['x-adplatform-timestamp'];

  // Verify signature
  if (!verifySignature(req.body, signature)) {
    return res.status(401).json({ error: 'Invalid signature' });
  }

  // Verify timestamp (prevent replay)
  const now = Math.floor(Date.now() / 1000);
  if (Math.abs(now - parseInt(timestamp)) > 300) {
    return res.status(401).json({ error: 'Request too old' });
  }

  const { event, data } = req.body;

  // Handle different event types
  switch (event) {
    case 'campaign.budget_threshold':
      console.log(`Campaign ${data.campaign_id} reached ${data.threshold}% budget`);
      // Send alert to advertiser
      break;

    case 'campaign.anomaly_detected':
      console.log(`Anomaly detected: ${data.description}`);
      // Create alert/ticket
      break;

    case 'conversion.received':
      console.log(`New conversion: ${data.conversion_value}`);
      // Update analytics
      break;
  }

  // Return 200 immediately
  res.status(200).json({ received: true });
});

app.listen(3001);
```

Testing & Debugging

Use Sandbox Environment

```
# Development
export API_BASE_URL=http://localhost:4000/api/v1

# Staging (for integration testing)
export API_BASE_URL=https://staging.adplatform.example.com/api/v1

# Production
export API_BASE_URL=https://adplatform.example.com/api/v1
```

Debugging Headers

Include diagnostic headers with requests:

```
curl -X GET https://adplatform.example.com/api/v1/campaigns/camp_456 \
-H "Authorization: Bearer ${TOKEN}" \
-H "X-Request-ID: debug-12345" \
-H "X-Debug: true"
```

The X-Request-ID appears in error responses and server logs, making it easier to track down issues.

Test Webhook Delivery

Use a webhook testing service like Webhook.cool:

1. Get a unique URL from webhook.cool
2. Register it with AdPlatform: POST /webhooks with your test URL
3. Trigger an event (e.g., reach budget threshold)
4. View webhook payload on webhook.cool

Log All Requests

```
// Middleware to log all API calls
fetch = (function(originalFetch) {
  return function(...args) {
    console.log(`[API] ${args[0]}`);
    return originalFetch.apply(this, args)
      .then(response => {
        console.log(`[API] ${response.status} ${args[0]}`);
        return response;
      });
  };
})(fetch);
```

API Reference

Authentication Endpoints

Endpoint	Method	Description
/oauth/authorize	GET	Initiate OAuth flow
/oauth/token	POST	Exchange code for token
/oauth/revoke	POST	Revoke token
/auth/verify-age	POST	Verify user age

Campaign Endpoints

Endpoint	Method	Description
/campaigns	POST	Create campaign
/campaigns	GET	List campaigns
/campaigns/:id	GET	Get campaign details
/campaigns/:id	PATCH	Update campaign
/campaigns/:id	DELETE	Delete campaign
/campaigns/:id/clone	POST	Clone campaign
/campaigns/:id/creatives	POST	Add creative
/campaigns/:id/creatives	GET	List creatives

Event Endpoints

Endpoint	Method	Description
/events/track	POST	Track single event
/events/track/batch	POST	Track multiple events
/pixel/:campaignId	POST	Conversion pixel

Analytics Endpoints

Endpoint	Method	Description
/analytics/dashboard/:campaignId	GET	Campaign dashboard
/analytics/creatives/:campaignId	GET	Creative performance
/analytics/export/:campaignId	GET	Export report
/analytics/heatmap/:campaignId	GET	Geographic heatmap

Webhook Endpoints

Endpoint	Method	Description
/webhooks	POST	Register webhook
/webhooks	GET	List webhooks
/webhooks/:id	DELETE	Delete webhook

Support

API Docs: <https://adplatform.example.com/api/docs>

API Status: <https://status.adplatform.example.com>

Slack Community: <https://adplatform-community.slack.com>

Email: api-support@adplatform.example.com

Last Updated: May 2026
Version: 1.0